DevOps_Day3-Disk-full

💾 **Day 3 — Production Incident Lab:** Disk Full (No space left on device)
Goal: Investigate and recover an EC2 instance where applications start failing because the root disk is at 100%.

Scenario

An application running on an **AWS EC2 (Ubuntu Linux)** instance begins to fail. Engineers see errors like:

- `No space left on device`

Typical symptoms in production:

- Applications stop writing logs
- Databases fail to write
- Web services crash or restart repeatedly
- SSH becomes slow or unreliable

Architecture (high level)

- User → HTTP request → Internet → EC2 (Ubuntu) → Application / Web Server

---

What you will practice

1. Confirm disk usage and impact
2. Identify which directory and files consume space
3. Apply a safe cleanup strategy
4. Verify the fix and add prevention steps

Step 1 — Confirm disk usage

Run:

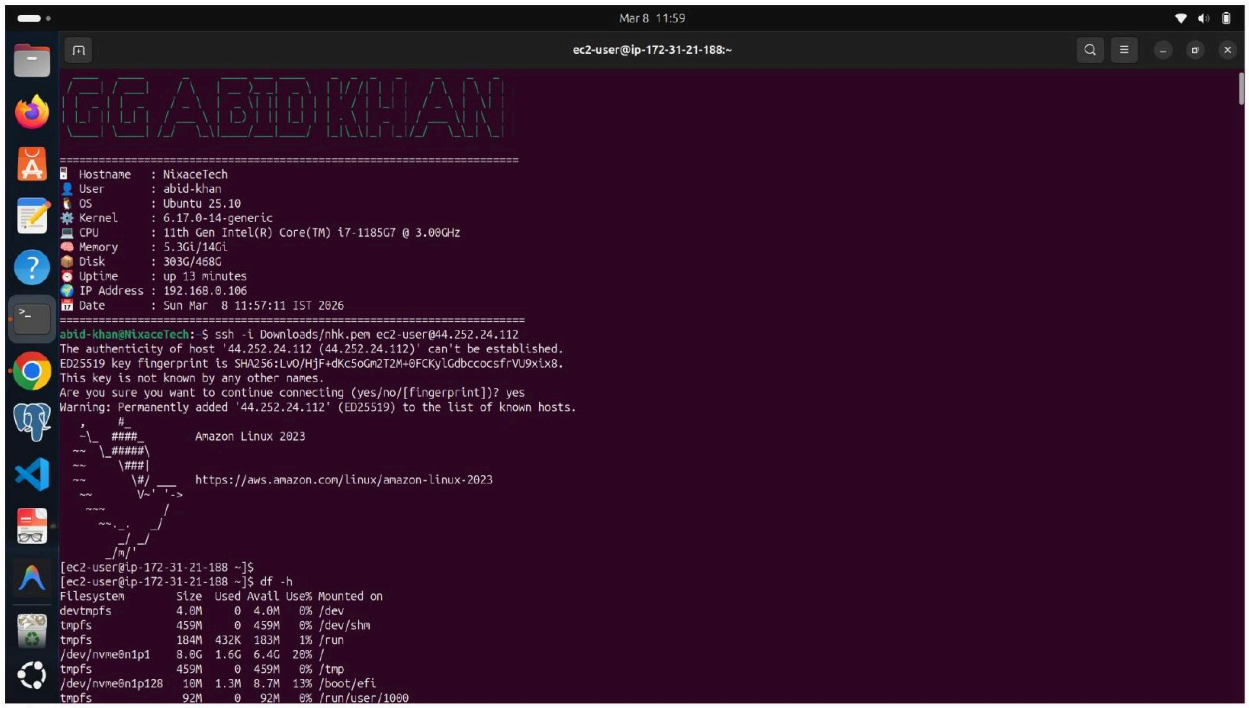
```
df -h
```

Example (disk full):

- `/dev/xvda1` shows **Use% = 100%**

Meaning:

- The instance has **no free space** for new files.
- Many services fail because they cannot write logs, temp files, or DB data.



Step 2 — Find which top-level directory is growing

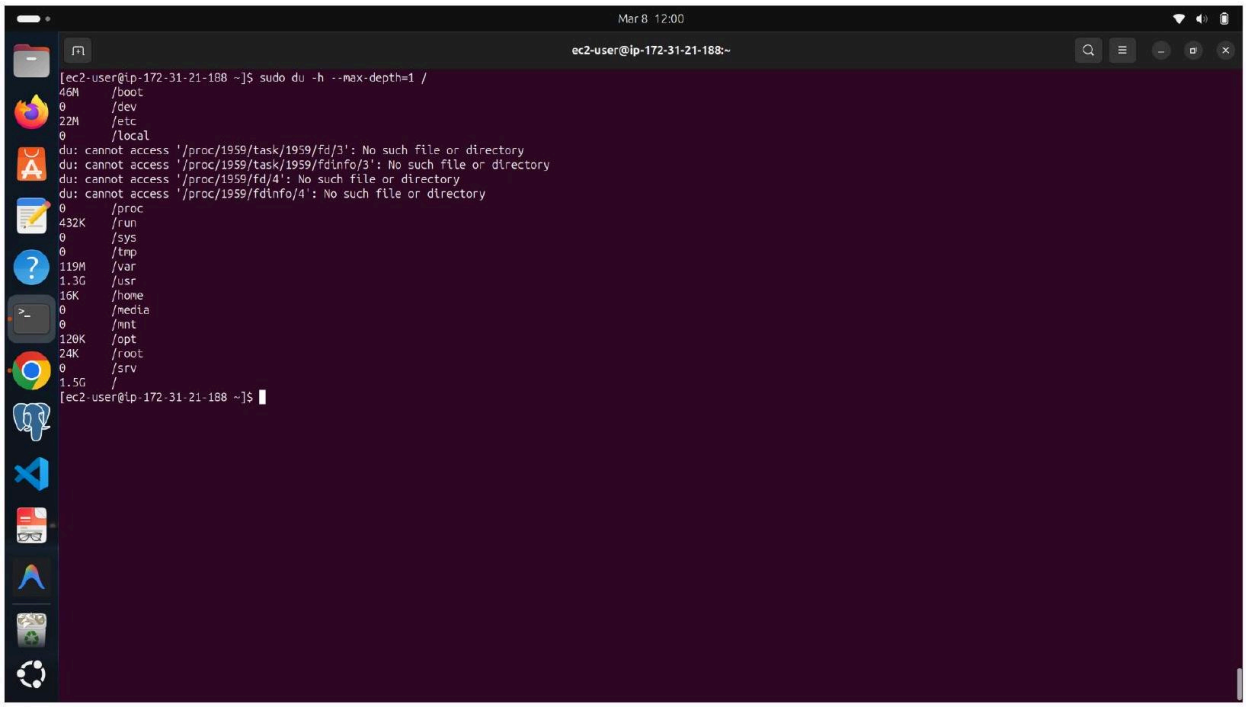
Run (top-level usage):

```
sudo du -h --max-depth=1 / 2>/dev/null
```

This shows which directories consume the most space.

Common “usual suspects”:

- `/var/log`
- `/var/lib`
- `/home`
- `/tmp`



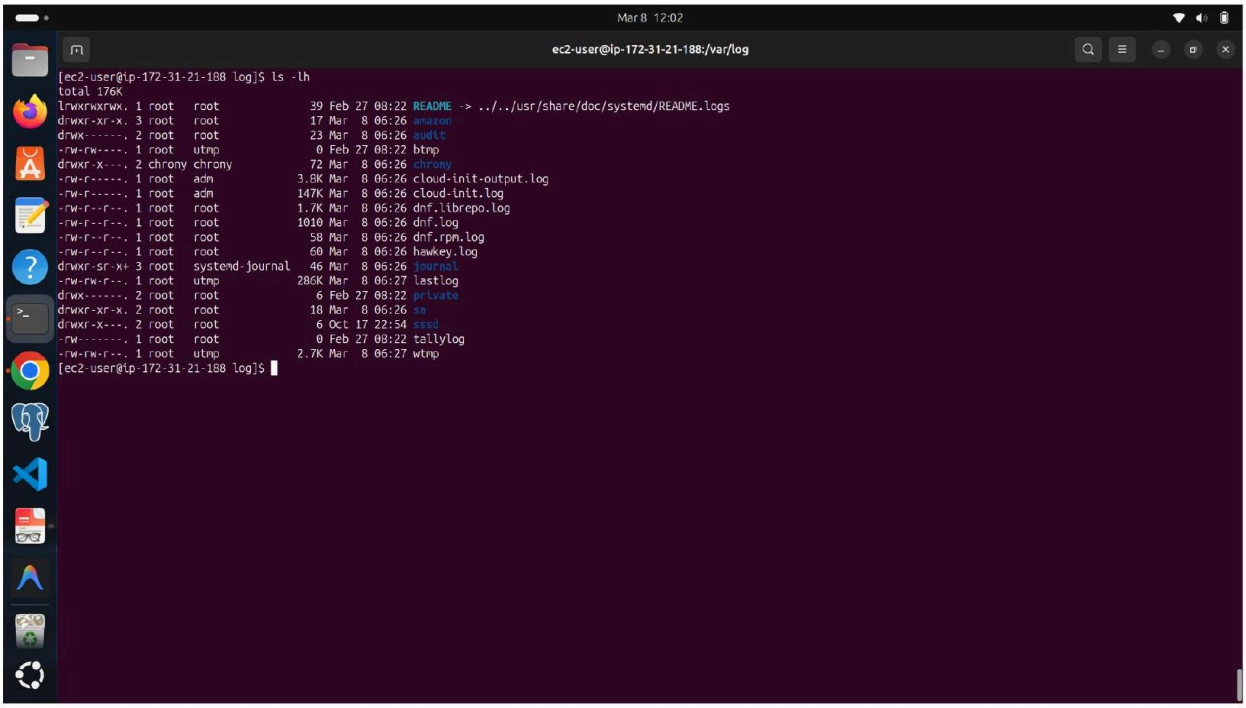
Step 3 — Investigate log growth (common root cause)

If `/var/log` is large:

```
cd /var/log
ls -lh
```

Look for unusually large log files such as:

- `syslog`
- `auth.log`
- Nginx access/error logs



Failure simulation (lab)

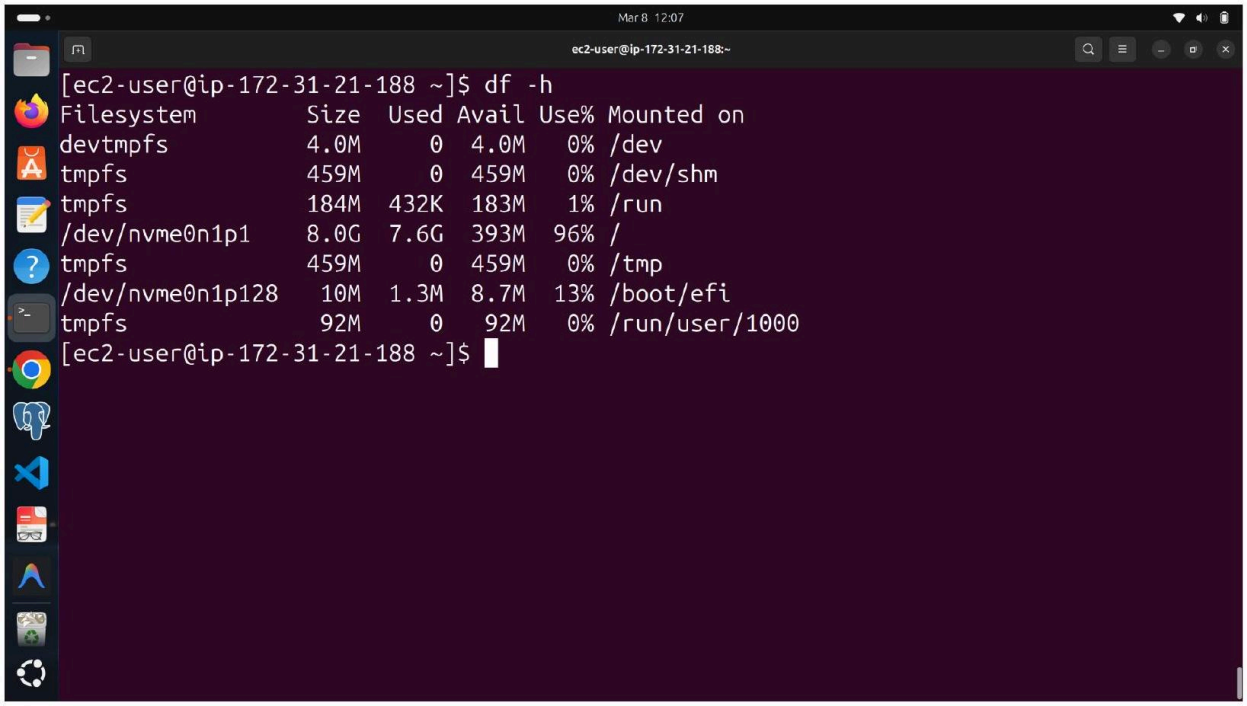
Simulate disk consumption by creating a large file:

```
fallocate -l 2G bigfile
```

Check again:

```
df -h
```

Expected: disk usage increases.



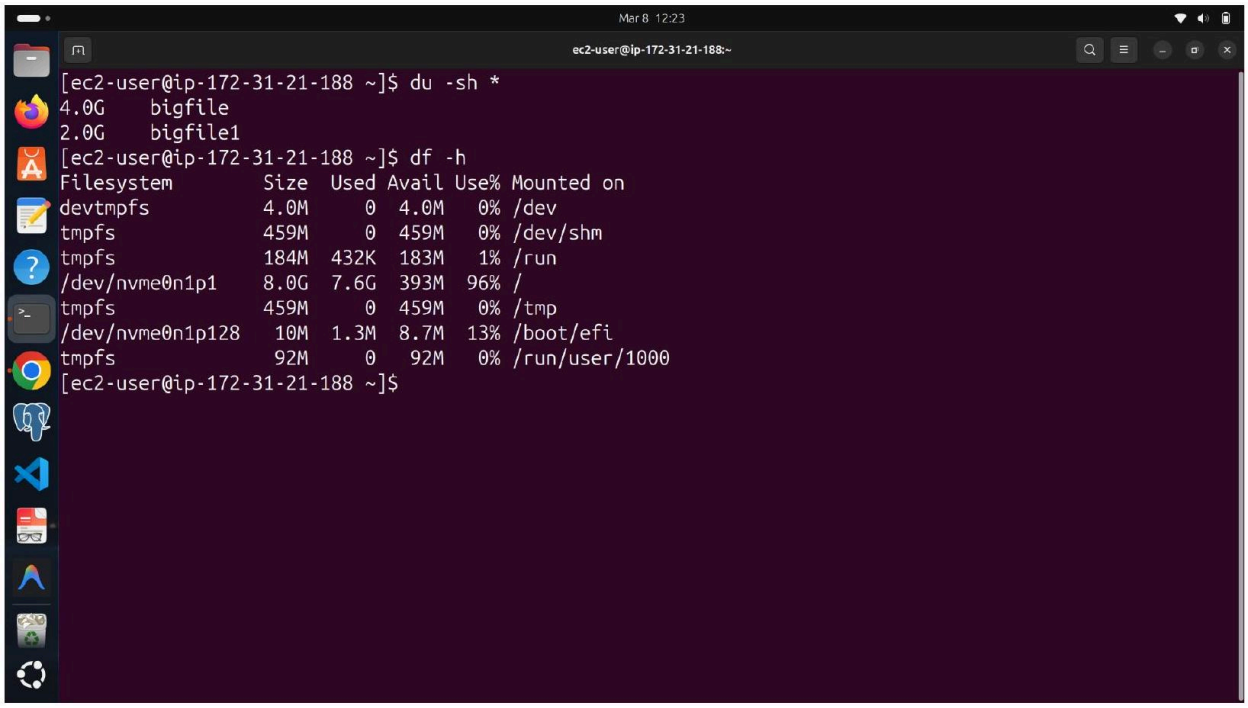
Investigation — Find the largest files

Option A (quick inside a directory):

```
sudo du -sh * | sort -h
```

Option B (find very large files globally):

```
sudo find / -type f -size +100M 2>/dev/null
```



Root cause (typical)

Disk became full due to one or more of these:

- Excessive log generation
- Missing or broken log rotation
- Large temporary files not cleaned up
- Application writing to disk unexpectedly

When disk reaches **100%**, applications often cannot:

- write logs
- create temporary files
- write database data

Resolution (safe cleanup)

Option 1 — Remove the lab file

```
rm -f bigfile
```

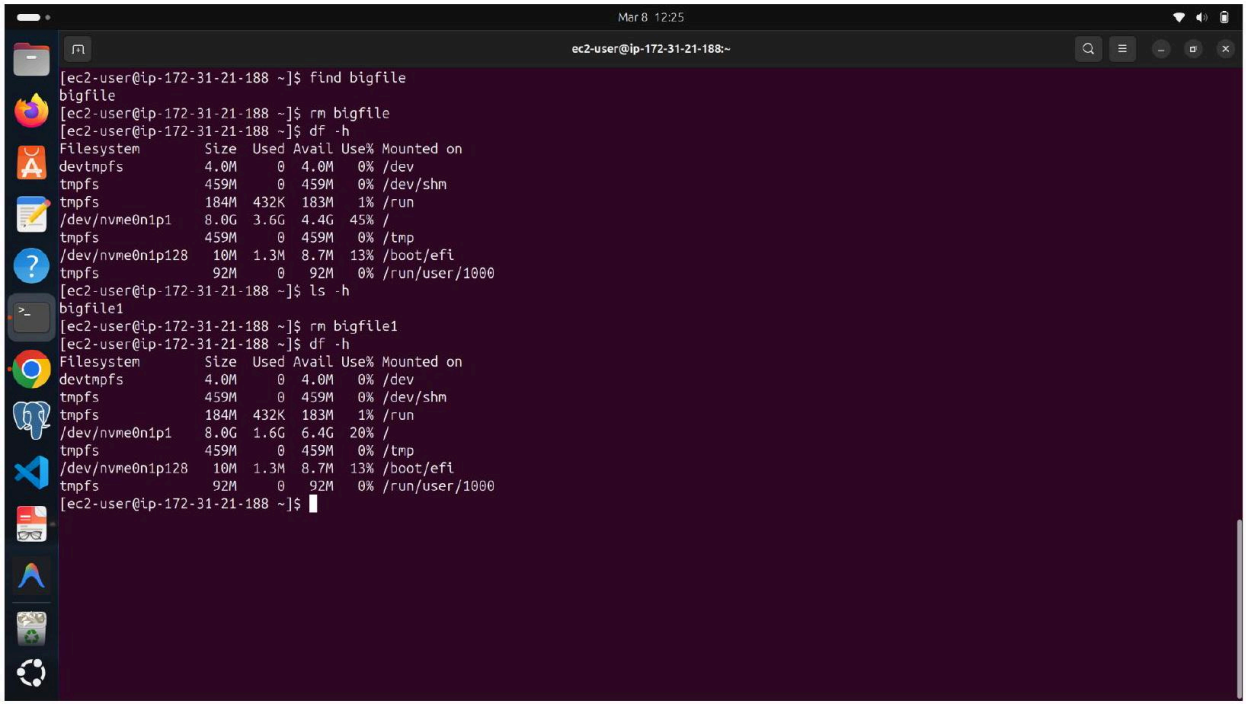
Option 2 — Truncate a large log file (keeps the file, clears contents)

Example:

```
sudo truncate -s 0 /var/log/syslog
```

Then verify free space:

```
df -h
```



Commands used (cheat sheet)

Command	Purpose
`df -h`	Check disk usage

Command	Purpose
`du -h --max-depth=1 /`	Find large directories
`find / -type f -size +100M`	Locate large files
`rm -f`	Delete files
`truncate -s 0`	Clear a log file safely

Lessons learned / prevention

- Add disk usage monitoring and alerts (for example, alert at 80% and 90%).
- Ensure **logrotate** is configured and running.
- Put application logs on a separate volume when needed.
- Set retention policies for logs and artifacts.

Interview questions (based on this lab) — with answers

Linux fundamentals

1. **What is the difference between `df` and `du`?**
   - `df`: reports free/used space at the filesystem level.
   - `du`: reports space used by files/directories.
2. **Why can a system behave oddly when the root disk hits 100%?**
   - Services cannot write logs, temp files, databases, or sockets, causing failures and instability.
3. **When would you use `truncate` instead of deleting a file?**
   - When a process is still writing to the file. Truncating keeps the same file handle/inode but frees space.

Practical troubleshooting

1. **Walk through your steps when you see `No space left on device`.**

- `df -h` to confirm which filesystem is full.
- `sudo du -h --max-depth=1 /` to find the biggest directory.
- `sudo find / -type f -size +100M 2>/dev/null` to identify large files.
- Clean safely (remove temp files, rotate logs, truncate huge logs).
- Re-check `df -h` and validate the application is healthy.

2. **If `/var/log` is growing fast, what are your next checks?**
   - Find the largest logs: `sudo du -sh /var/log/* | sort -h`.
   - Identify the service generating logs (app errors, traffic spike, debug logging enabled).
   - Verify and fix log rotation.
3. **How do you find the top 10 largest files on a server?**
   - `sudo du -ahx / 2>/dev/null | sort -rh | head -n 10`

Reliability and prevention

1. **What monitoring would you set up to catch disk issues early?**
   - Alerts at 80% (warning) and 90% (critical) for disk usage.
   - Inode monitoring with `df -i`.
   - Growth-rate alerts for key directories like `/var/log`.
2. **What is log rotation, and how would you validate it is working?**
   - Rotation compresses/archives old logs and enforces retention.
   - Validate with `sudo logrotate -d /etc/logrotate.conf` and by checking rotated files in `/var/log`.
3. **How would you prevent one noisy application from filling the entire root disk?**
   - Set retention and rotation.
   - Reduce log level and fix root-cause errors.

- Put logs on a separate volume/partition or ship logs off-host (central logging).